

IRCTC Anti-Bot Detection System - Patent Claims & Implementation Guide

Executive Summary

This document outlines a novel AI/ML-based anti-bot detection system specifically designed for railway ticket booking platforms like IRCTC. The system combines behavioral biometrics, temporal pattern analysis, and multi-modal authentication to detect and prevent automated bot-based ticket booking.

10 Novel Patentable Claims

1. Multi-Temporal Behavioral Fingerprinting System

Claim: A system for detecting automated booking agents by analyzing micro-temporal patterns in user interactions, including keystroke dynamics, mouse movement velocity curves, scroll acceleration patterns, and form-filling rhythm signatures that distinguish human users from scripted bots. The system creates a unique behavioral fingerprint with temporal resolution under 10 milliseconds.

Industrial Application: Real-time detection during Tatkal booking hours without degrading user experience.

2. Adaptive Session Entropy Analysis Framework

Claim: A method utilizing information entropy calculations across multiple session parameters (click sequence unpredictability, navigation path randomness, timing jitter) to compute a real-time "human-likeness score." Bot behavior exhibits unnaturally low entropy due to programmatic consistency.

Industrial Application: Identifies bot patterns even when bots randomize individual features but maintain low overall behavioral entropy.

3. Pre-Booking Intent Detection via Weighted Clickstream Analysis

Claim: An AI system that analyzes pre-booking clickstream patterns using graph neural networks to detect pre-filled form submissions characteristic of bot software like BraHmos. The system identifies suspicious patterns when users skip typical browsing/search phases and directly proceed to booking with pre-populated data.

Industrial Application: Flags accounts that show patterns of preparation consistent with automated booking tools.

4. Cascading Multi-Factor CAPTCHA Challenge System

Claim: A dynamic CAPTCHA system that adjusts challenge difficulty and type based on real-time risk scores. For high-risk sessions, the system deploys cascading challenges including image recognition, temporal puzzles, and context-aware questions about journey details that require genuine booking knowledge.

Industrial Application: Defeats automated CAPTCHA solvers by introducing unpredictable, context-specific challenges.

5. Device-Browser-Network Fingerprinting Ensemble

Claim: A comprehensive fingerprinting method combining device characteristics (screen resolution, installed fonts, WebGL renderer), browser properties (plugins, timezone, language), network metadata (IP geolocation consistency, VPN detection, latency patterns), and historical behavior to create multi-dimensional user profiles that are extremely difficult for bots to spoof consistently.

Industrial Application: Identifies bot farms and multiple fake accounts from same infrastructure.

6. OTP Timing Pattern Analysis for Auto-Fill Detection

Claim: A system analyzing temporal patterns between OTP generation and submission to detect automated OTP reading. The method identifies statistically improbable speeds (under 2 seconds) and consistent timing signatures that indicate programmatic OTP capture and auto-submission.

Industrial Application: Directly targets BraHmos-style bots that auto-read and submit OTPs.

7. Ensemble Learning Framework with Adversarial Robustness

Claim: A machine learning ensemble combining Random Forest, XGBoost, LightGBM, and Deep Neural Networks with adversarial training to resist bot evasion techniques. The system includes a meta-learner that weighs individual model predictions based on recent attack patterns observed in production.

Industrial Application: Maintains high accuracy even as bots evolve their strategies to evade detection.

8. Real-Time Graph-Based Account Clustering

Claim: A graph neural network system that builds real-time relationship graphs between user accounts based on shared attributes (IP addresses, payment methods, passenger details, booking patterns) to identify coordinated bot networks. The system detects communities of accounts exhibiting synchronized booking attempts.

Industrial Application: Identifies ticket tout networks operating multiple coordinated bot accounts.

9. Biometric-Enhanced Authentication for High-Value Transactions

Claim: A system requiring biometric verification (Aadhaar-based face recognition, fingerprint scan via mobile) for Tatkal bookings or when risk scores exceed threshold. The system integrates with India Stack/DigiLocker for seamless verification while maintaining privacy through federated authentication.

Industrial Application: Makes bot-based booking economically unfeasible for high-demand routes.

10. Continuous Learning System with Human-in-the-Loop Validation

Claim: An adaptive ML pipeline that continuously retrain models using new bot patterns detected by security analysts, incorporates user appeals and false positive corrections, and deploys updated models with A/B testing.

The system includes confidence scoring for borderline cases requiring human review.

Industrial Application: Ensures system evolves with emerging bot techniques while maintaining fairness through human oversight.

Recommended Open Source Kaggle Datasets

Primary Datasets

1. Bot Detection Datasets

- **Kaggle Bot Account Detection** ([shriyashjagtap/kaggle-bot-account-detection](#))
 - 799 users (400 bots, 399 real users)
 - Features: account age, activity patterns, comment frequency
- **Twitter Bot Detection** ([goyaladi/twitter-bot-detection-dataset](#))
 - ~37,000 accounts with bot/human labels
 - Features: posting frequency, follower patterns, account metadata

2. Clickstream & Behavioral Datasets

- **E-commerce Clickstream Dataset** ([waqi786/e-commerce-clickstream-and-transaction-dataset](#))
 - User interaction sequences, session data
 - Temporal patterns, navigation paths
- **E-Shop Clickstream** ([arashnic/e-shop-clickstream-dataset](#))
 - Page views, click sequences, session analytics
 - Bounce rates, cart abandonment patterns

3. Network Traffic & Anomaly Detection

- **IoT Botnets Attack Detection** ([saurabhshahane/anomaly-detection-using-deep-learning](#))
 - Network-level bot behavior
 - Useful for IP pattern analysis

Synthetic Data Generation

Since IRCTC-specific data isn't publicly available, you'll need to:

- **Generate synthetic booking sessions** with tools like Faker
- **Simulate bot patterns** based on documented BraHmos behavior
- **Create labeled training data** with known bot vs. human sessions

Implementation in VS Code with GitHub Copilot

Setup Commands

```
bash

# Create project structure
mkdir irtc-antibot-detection
cd irtc-antibot-detection

# Create virtual environment
python -m venv venv
source venv/bin/activate # On Windows: venv\Scripts\activate

# Install dependencies
pip install numpy pandas scikit-learn xgboost lightgbm tensorflow keras matplotlib seaborn jupyter

# Create Jupyter notebook
jupyter notebook irtc_antibot_model.ipynb
```

GitHub Copilot Prompts for Model Development

1. Data Loading & Preprocessing

```
python

# Prompt for Copilot:
# Load bot detection dataset from Kaggle, handle missing values,
# encode categorical features, and split into train/test sets with 80/20 ratio.
# Include feature engineering for temporal patterns and session statistics.
```

2. Feature Engineering

```
python
```

```
# Prompt for Copilot:  
# Create advanced features for bot detection including:  
# - Session entropy calculation based on click sequences  
# - Keystroke dynamics features (typing speed, pause patterns)  
# - Mouse movement features (velocity, acceleration, trajectory smoothness)  
# - Temporal features (time between actions, session duration)  
# - Device fingerprint features (browser, OS, screen resolution)  
# - Network features (IP entropy, geolocation consistency)  
# Use pandas and numpy for efficient computation.
```

3. Exploratory Data Analysis

```
python  
  
# Prompt for Copilot:  
# Create comprehensive EDA for bot vs human users including:  
# - Distribution plots for all numerical features  
# - Correlation heatmap  
# - Time-series analysis of booking patterns  
# - Statistical tests for feature significance  
# Use seaborn and matplotlib for visualization.
```

4. Model Architecture - Random Forest

```
python  
  
# Prompt for Copilot:  
# Build a Random Forest classifier for bot detection with:  
# - Hyperparameter tuning using GridSearchCV  
# - Feature importance analysis  
# - Cross-validation with 5 folds  
# - ROC-AUC evaluation  
# - Confusion matrix visualization  
# Optimize for high precision to minimize false positives.
```

5. Model Architecture - XGBoost

```
python
```

```
# Prompt for Copilot:  
# Implement XGBoost classifier with:  
# - Early stopping to prevent overfitting  
# - SMOTE for handling class imbalance  
# - Bayesian optimization for hyperparameter tuning  
# - SHAP values for model interpretability  
# - Learning curve analysis
```

6. Model Architecture - Deep Neural Network

```
python  
  
# Prompt for Copilot:  
# Create a deep neural network using TensorFlow/Keras for bot detection:  
# - Input layer matching feature dimensions  
# - 3 hidden layers with batch normalization and dropout  
# - Binary classification output with sigmoid activation  
# - Use Adam optimizer with learning rate scheduling  
# - Implement early stopping and model checkpointing  
# - Add class weights to handle imbalance  
# - Plot training history (loss and accuracy curves)
```

7. Ensemble Model

```
python  
  
# Prompt for Copilot:  
# Build an ensemble model combining Random Forest, XGBoost, and Neural Network:  
# - Implement soft voting classifier  
# - Weighted averaging based on individual model performance  
# - Stack models using logistic regression as meta-learner  
# - Compare ensemble performance against individual models  
# - Generate comprehensive classification report
```

8. Real-Time Inference Pipeline

```
python
```

```
# Prompt for Copilot:
# Create a real-time bot detection inference pipeline:
# - Load pre-trained ensemble model
# - Implement feature extraction from live session data
# - Calculate risk score (0-100 scale)
# - Define decision thresholds (low/medium/high risk)
# - Add logging for audit trail
# - Return JSON response with prediction and confidence
# Include error handling and input validation.
```

9. Model Evaluation & Metrics

```
python

# Prompt for Copilot:
# Comprehensive model evaluation including:
# - Precision, Recall, F1-Score for each class
# - ROC curve and AUC score
# - Precision-Recall curve
# - Confusion matrix with percentages
# - Classification report
# - False Positive Rate and False Negative Rate analysis
# - Cost-sensitive evaluation (weight FP vs FN differently)
# Use sklearn.metrics for all calculations.
```

10. Deployment Preparation

```
python

# Prompt for Copilot:
# Prepare model for production deployment:
# - Serialize trained models using joblib
# - Create model versioning system
# - Build Flask/FastAPI REST API endpoint
# - Implement request/response validation with pydantic
# - Add rate limiting and authentication
# - Create Docker container with all dependencies
# - Write unit tests for critical functions
# Include API documentation with example requests.
```

Complete Project Structure

```
irctc-antibot-detection/
├── data/
│   ├── raw/           # Original datasets from Kaggle
│   ├── processed/     # Cleaned and feature-engineered data
│   └── synthetic/     # Generated synthetic bot patterns
├── notebooks/
│   ├── 01_data_exploration.ipynb
│   ├── 02_feature_engineering.ipynb
│   ├── 03_model_training.ipynb
│   └── 04_model_evaluation.ipynb
├── src/
│   ├── data_preprocessing.py
│   ├── feature_engineering.py
│   ├── model_training.py
│   ├── model_inference.py
│   └── utils.py
├── models/
│   ├── random_forest_v1.pkl
│   ├── xgboost_v1.pkl
│   ├── neural_net_v1.h5
│   └── ensemble_v1.pkl
├── api/
│   ├── app.py         # FastAPI application
│   ├── schemas.py     # Request/response models
│   └── dependencies.py
├── tests/
│   ├── test_preprocessing.py
│   ├── test_models.py
│   └── test_api.py
├── requirements.txt
├── Dockerfile
├── README.md
└── .gitignore
```

Key Technical Innovations for Patent Strength

1. **Novel Feature Engineering:** Micro-temporal behavioral patterns at <10ms resolution
 2. **Domain-Specific Architecture:** Railway booking-specific features (route popularity, Tatkal timing patterns)
 3. **Adversarial Robustness:** Specific defense against documented bot techniques (BraHmos, Tesla, Nexus)
 4. **Multi-Modal Fusion:** Combining behavioral, network, device, and biometric signals
 5. **Scalable Real-Time Processing:** Designed for millions of concurrent Tatkal requests
 6. **Explainable AI:** SHAP/LIME integration for regulatory compliance and user transparency
 7. **Privacy-Preserving:** Federated learning approach for biometric data
 8. **Continuous Adaptation:** AutoML pipeline that evolves with new bot patterns
-

Industrial Deployment Considerations

Scalability

- Handles 60,000+ booking requests/minute during peak Tatkal hours
- Distributed architecture using Kubernetes for horizontal scaling
- Redis caching for real-time feature computation

Performance

- Prediction latency: <50ms per session
- Model update frequency: Daily retraining with new labeled data
- 99.9% uptime SLA for critical booking periods

Regulatory Compliance

- GDPR/Data Protection Act compliance for user data
- Explainable predictions for user appeals
- Audit logs for all bot detection decisions
- Integration with existing IRCTC Aadhaar verification

Business Impact

- **Reduction in Fake IDs:** Target 90%+ reduction (IRCTC blocked 35M fake IDs in 2024)
 - **Fair Access:** Equal opportunity for genuine passengers
 - **Revenue Protection:** Prevent ticket touting and black market sales
 - **System Performance:** Reduce server load from bot traffic
-

Next Steps

1. **Dataset Preparation:** Download and preprocess Kaggle datasets
 2. **Baseline Model:** Implement simple Random Forest classifier
 3. **Feature Engineering:** Add domain-specific railway booking features
 4. **Ensemble Development:** Combine multiple models for robust detection
 5. **Validation:** Test against synthetic BraHmos-like bot patterns
 6. **Patent Filing:** Work with patent attorney to file claims
 7. **Pilot Testing:** Deploy in controlled environment with IRCTC
 8. **Production Rollout:** Gradual deployment with monitoring
-

Success Metrics

- **Detection Rate:** >95% bot detection accuracy
 - **False Positive Rate:** <2% (minimize genuine user friction)
 - **Response Time:** <100ms for real-time scoring
 - **Scalability:** Handle 10x traffic spike during festivals
 - **Adaptability:** Detect new bot patterns within 48 hours of emergence
-

This system represents a comprehensive, industrially viable solution to the critical problem of automated ticket booking fraud in India's railway system.